

100 Days of Azure DevOps

Day 57 Handout — CD to Kubernetes without Git is sticky-note kubectl wearing a hoodie

6 - Containers & Kubernetes · Personal lab only · Educational content · Not a sales pitch

Architecture A — laptop kubectl vs KubernetesManifest@1 (same YAML, different source of truth)

Dimension	kubectl apply from PC	Pipeline deploy
Source	Whatever is on disk, maybe uncommitted	k8s/*.yaml (or baked Helm) on the commit being built
Identity	Your user kubeconfig. Easy to be cluster-admin	Azure RM / Kubernetes service connection. Scope it; Day 63/69 will be rude
Image	Easy to leave :latest	KubernetesManifest@1 containers: input substitutes Build.BuildId
Namespace	You remember -n, or you don't	inputs.namespace: workloads — never kube-system
No AKS	kind apply is still Git if the files are committed	Same YAML, local kubeconfig job or skip Azure target honestly

Architecture B — decide GitOps later vs push-from-pipeline today

Constraint	Default today	Change when
Process	Push CD: pipeline applies manifests from Git	Flux/Argo (pull GitOps) is a later day — do not fake it with a blog title
Cluster missing	Apply the same files to kind; keep them in Git	Creating AKS today without a destroy date — still no
Tag move	Substitute image to <acr>/myapp:\$(Build.BuildId)	Manifest frozen on latest is not a deploy of this build
RBAC	Connection can deploy to namespace workloads	cluster-admin service connection as a lab convenience is a finding
Helm	Manifests today; Helm upgrade is Day 58	Do not copy three environment folders of YAML this afternoon

Architecture C — pitfalls that apply Friday's cluster with Thursday's Git

Pitfall	What you see	Fix
Sticky-note apply	Three people, three tags, one cluster	Pipeline is the only apply to that cluster
Wrong namespace	Pods in default; Service in workloads; empty endpoints	namespace: workloads on every object and on the task
Image not substituted	Old digest still running	containers: input or a bake step; kubectl set image is not the process
kube-system deploy	Task namespace left default/system	Fail the review. Plumbing namespace is not an app target
cluster-admin forever	Service connection can delete the cluster	Namespace-scoped Role later; today at least not Owner on the subscription

One-liner to remember

If Git does not know the cluster state you intended, you do not have CD. You have a memory. Hoodies are not a release strategy.

Step-by-step lab (personal Azure / Azure DevOps only)

Personal Azure DevOps project azure-100-labs. Target AKS if it exists with a destroy date, otherwise kind. Manifests in Git.

- 1 Commit k8s/*.yaml from Day 55 (namespace workloads). Confirm no image: ...:latest as the only tag — use a placeholder the task will replace, or the Build.BuildId you already pushed.
- 2 If AKS exists: Project Settings → Service connections → Azure Resource Manager (or Kubernetes) to the personal cluster. Note the role. Not cluster-admin if you can avoid it.
- 3 Add a pipeline job after Docker@2 (or a second pipeline) with KubernetesManifest@1 action: deploy, namespace workloads, manifests k8s/*.yaml, containers: <acr>.azurecr.io/myapp:\${Build.BuildId}.
- 4 Run it. kubectl -n workloads get deploy,po,svc and kubectl -n workloads describe po | find the image. The running tag must be this Build.BuildId.
- 5 If no AKS: kubectl apply -f k8s/ against kind from the same files, and still keep a YAML pipeline that would deploy — identity is the only missing piece. Do not treat laptop kubectl as the production process.
- 6 Write docs/aks-deploy-day57.md: connection name, namespace, image tag that actually ran. Helm is tomorrow for values; do not fork the YAML per mood.

Starter YAML — KubernetesManifest@1 substitutes the image

```
- task: KubernetesManifest@1
  displayName: Deploy to workloads (not kube-system)
  inputs:
    action: deploy
    connectionType: Azure Resource Manager
    azureSubscriptionConnection: aks-connection
    azureResourceGroup: rg-day56
    kubernetesCluster: aks-lab
    namespace: workloads
    manifests: k8s/*.yaml
    containers: |
      <uniqueacr>.azurecr.io/myapp:${Build.BuildId}
```

Done checklist

- Manifests in Git are the source; namespace is workloads
- Running image tag equals this build (or kind apply of the same files)
- Did not kubectl apply from a dirty laptop as the CD story
- Service connection noted; not treated as cluster-admin folklore

Tomorrow — Day 58

Helm charts — values.yaml instead of YAML-mountain copy-paste